

PROCESSO DE DESENVOLVIMENTO DE SOFTWARE



Diego Augusto Barros
ENSINAR É TUDO O QUE SE PODE

CONTEÚDO

1. Processo de Desenvolvimento de Software
2. O Framework de Processo
3. Modelos de Processos de Software



1

PROCESSO DE DESENVOLVIMENTO DE SOFTWARE

INTRODUÇÃO

Um **processo de engenharia** consiste em um conjunto de **atividades inter-relacionadas** que **transformam** um ou mais **insumos em produtos**, enquanto **consomem recursos** para realizar essa transformação.



INTRODUÇÃO

Muitos processos em disciplinas tradicionais de engenharia, tais como: elétrica, mecânica, civil e química;

Frequentemente envolvem a transformação de energia e entidades físicas de uma forma para outra.



INTRODUÇÃO

Uma represa hidrelétrica que converte energia potencial em energia elétrica



Fonte da Imagem: Click Foz – clickfozdoiguacu.com.br

INTRODUÇÃO

Uma refinaria de petróleo que utiliza processos químicos para transformar petróleo bruto em gasolina.



Fonte da Imagem: Programa de Aceleração do Crescimento no Flickr.com

PROCESSOS DE ENGENHARIA DE SOFTWARE

Os **processos de engenharia de software** estão relacionados às **atividades** realizadas por engenheiros de software **para desenvolver, manter e operar software**, tais como:

- Requisitos,
- Design,
- Construção,
- Teste,
- Gerenciamento de Configuração e outros processos de engenharia de software.



Para facilitar a leitura e entendimento,
Processo de Engenharia de Software
será chamado de **Processo de Software**

Um **processo de software** é um
conjunto de **atividades relacionadas** que
levam à **produção** de um produto de
software.

NÃO CONFUNDA

Processo de software refere-se a atividades de trabalho, não ao processo de execução do software implementado.

PROCESSO DE SOFTWARE

As atividades do Processo de Software podem envolver o desenvolvimento de um sistema:

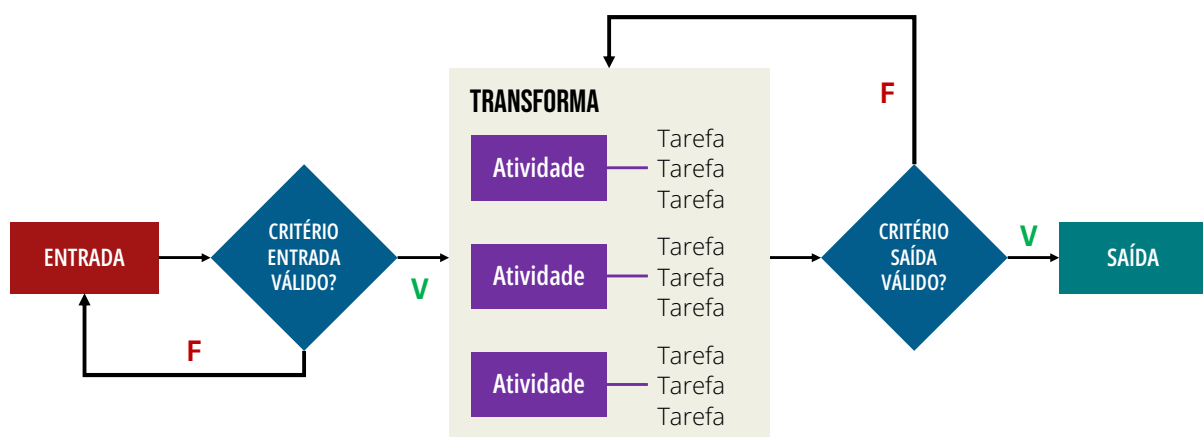
1. A partir do zero em uma linguagem padrão de programação como Java ou C;
2. Através da extensão e modificação de sistemas existentes;
3. Por meio da configuração e integração de softwares de prateleira ou componentes do sistema.

OBJETIVOS DO PROCESSO DE SOFTWARE

- Facilitar a compreensão humana, comunicação e coordenação;
- Auxiliar na gestão de projetos de software;
- Medir e melhorar a qualidade dos produtos de software de maneira eficiente;
- Apoiar a melhoria de processos;
- Fornecer uma base para o suporte automatizado da execução de processos.

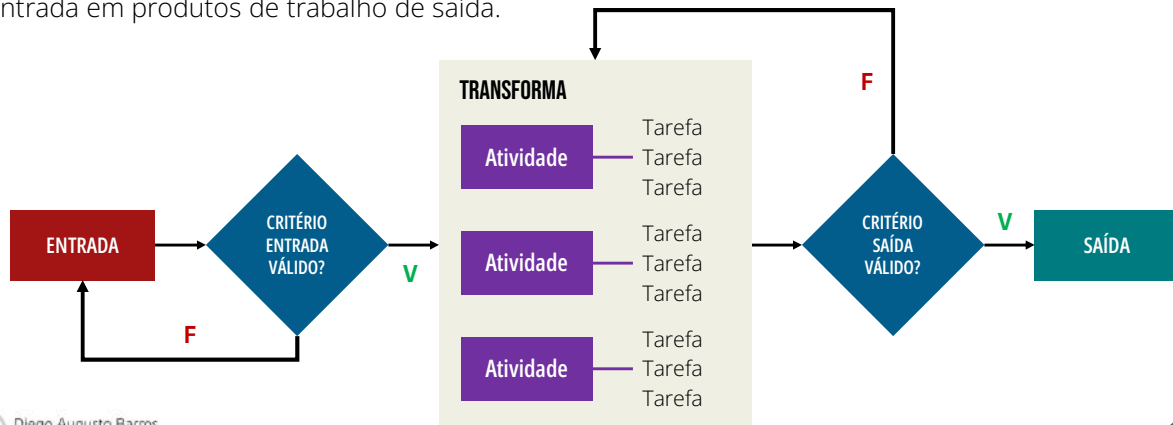


ELEMENTOS DE UM PROCESSO DE SOFTWARE



ELEMENTOS DE UM PROCESSO DE SOFTWARE

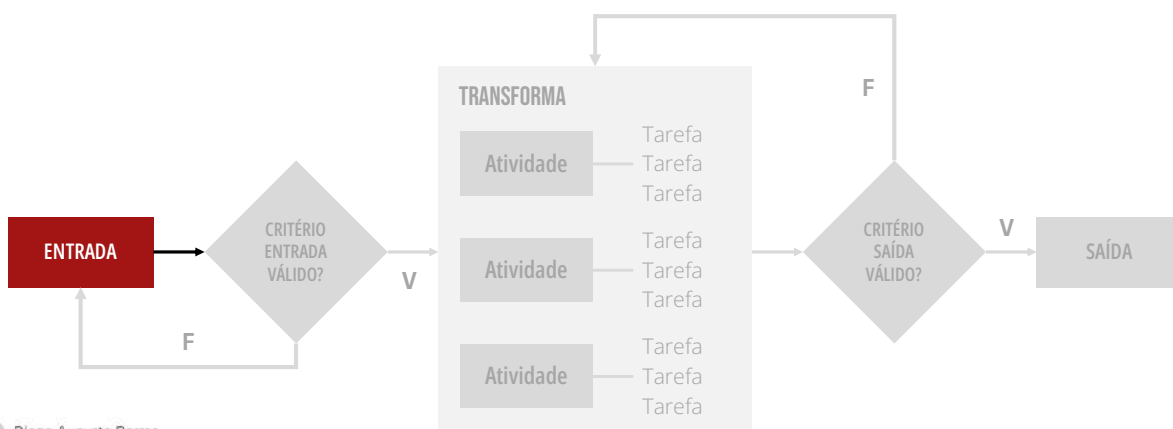
Um processo de software é um conjunto de atividades e tarefas inter-relacionadas que transformam produtos de trabalho de entrada em produtos de trabalho de saída.



ELEMENTOS DE UM PROCESSO DE SOFTWARE

Uma entrada do processo pode ser

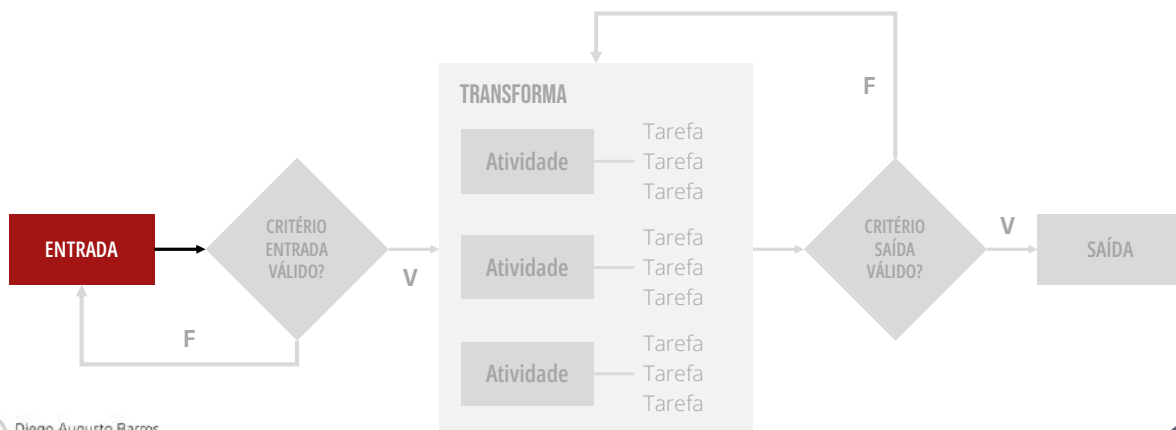
- Um evento desencadeante;
- A saída de outro processo.



ELEMENTOS DE UM PROCESSO DE SOFTWARE

A descrição de um processo de software inclui

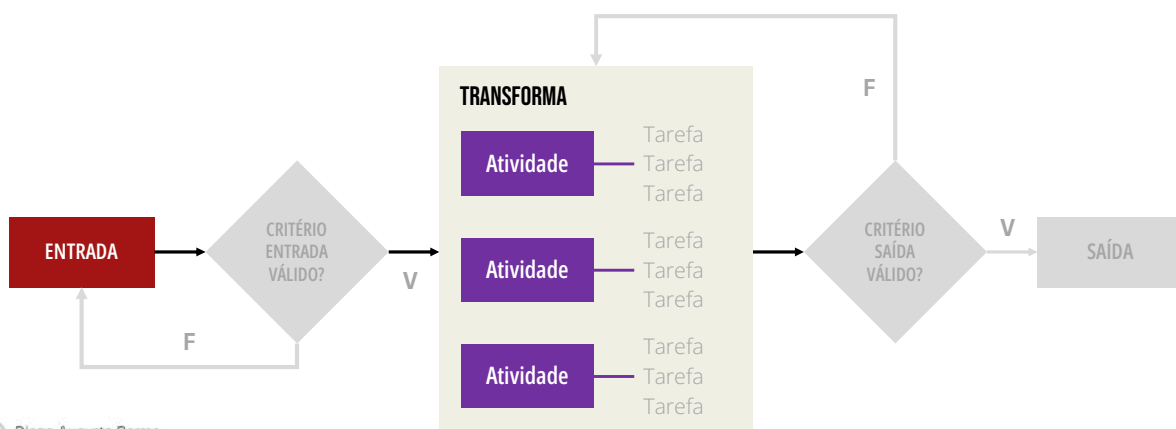
- Entradas (*inputs*) necessárias;



ELEMENTOS DE UM PROCESSO DE SOFTWARE

A descrição de um processo de software inclui

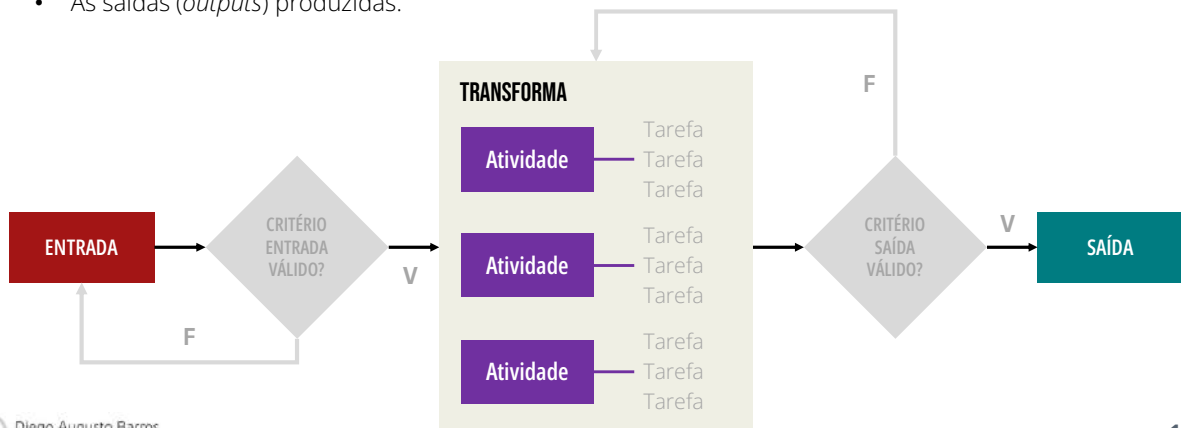
- Entradas (*inputs*) necessárias;
- Atividades de trabalho de transformação;



ELEMENTOS DE UM PROCESSO DE SOFTWARE

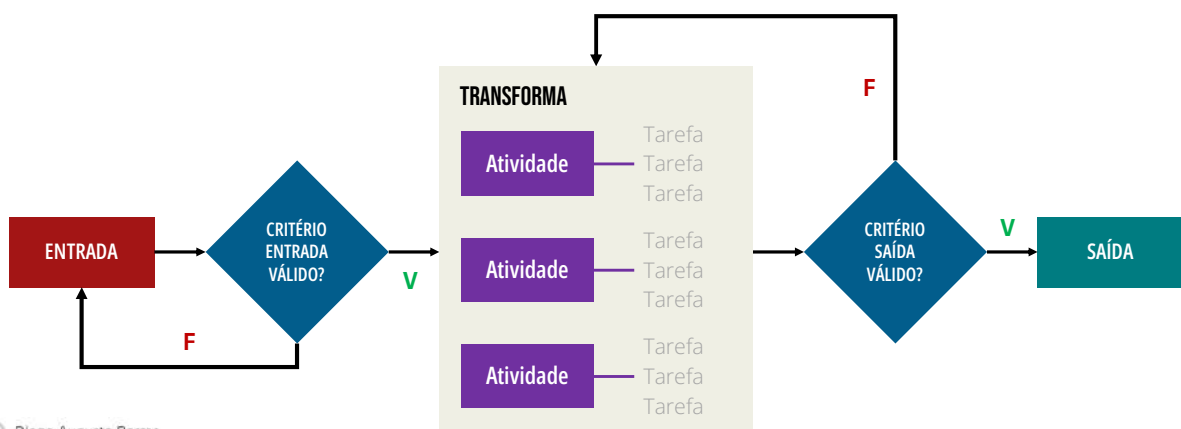
A descrição de um processo de software inclui

- Entradas (*inputs*) necessárias;
- Atividades de trabalho de transformação;
- As saídas (*outputs*) produzidas.



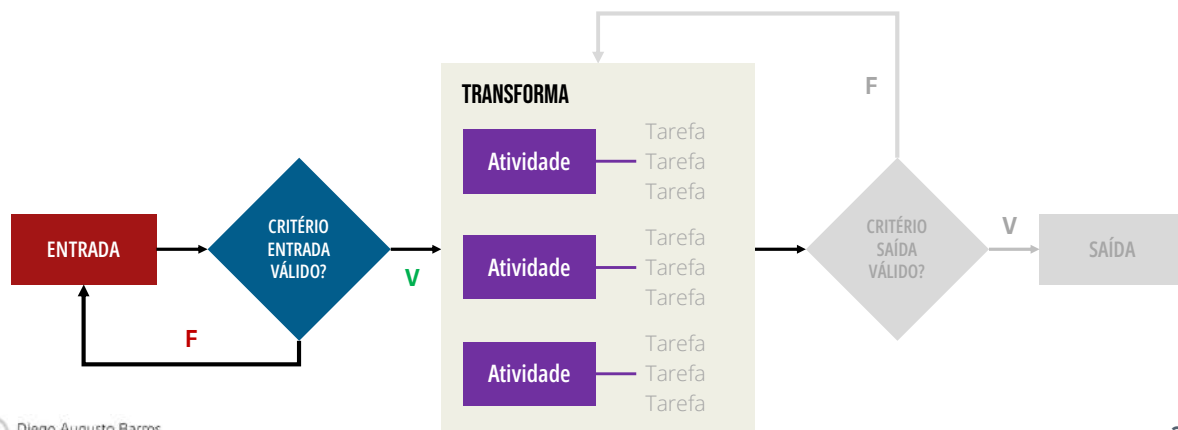
ELEMENTOS DE UM PROCESSO DE SOFTWARE

Um processo de software pode também incluir seus critérios de entrada e saída



ELEMENTOS DE UM PROCESSO DE SOFTWARE

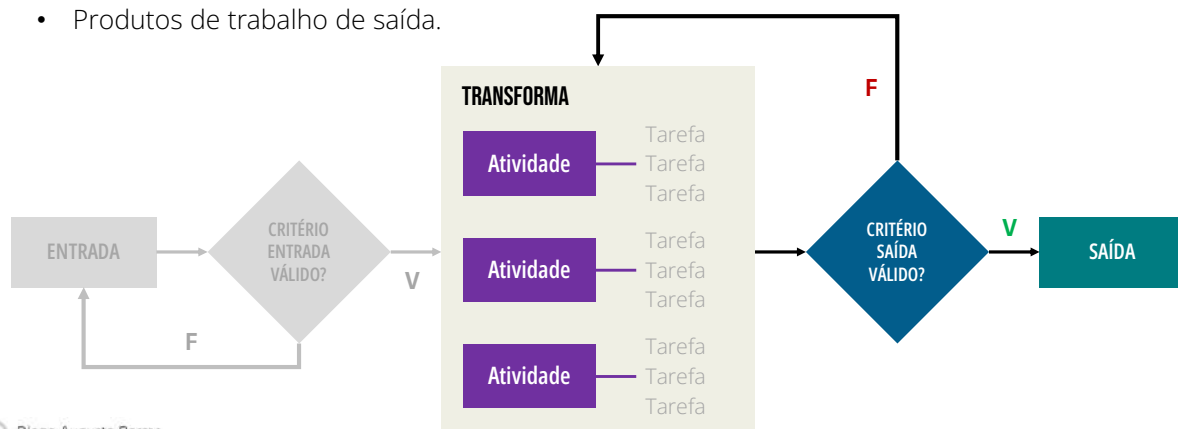
Os critérios de entrada devem ser satisfeitos antes que um processo possa começar.



ELEMENTOS DE UM PROCESSO DE SOFTWARE

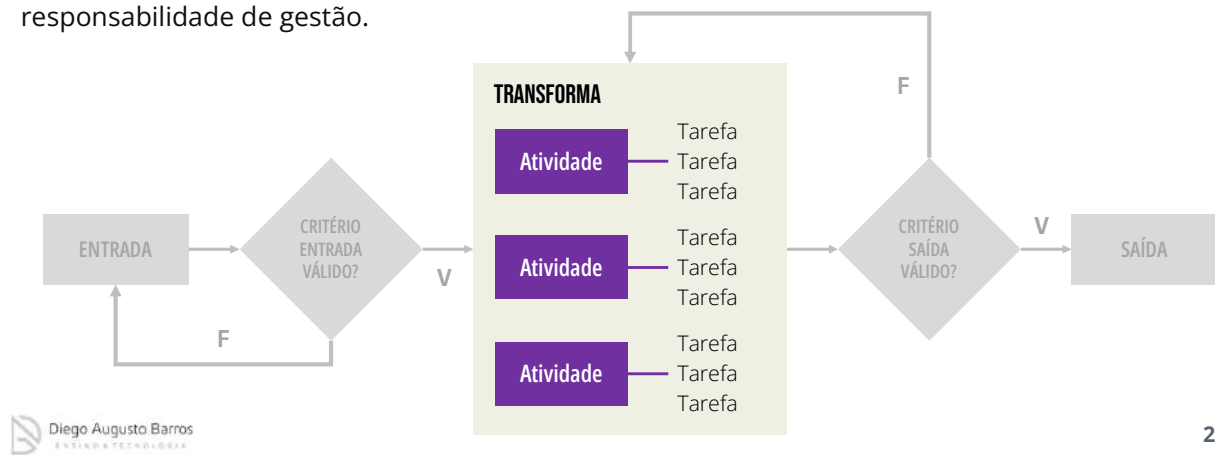
Todas as condições especificadas devem ser atendidas antes que um processo possa ser concluído com sucesso, incluindo:

- Os critérios de aceitação para o produto;
- Produtos de trabalho de saída.



ELEMENTOS DE UM PROCESSO DE SOFTWARE

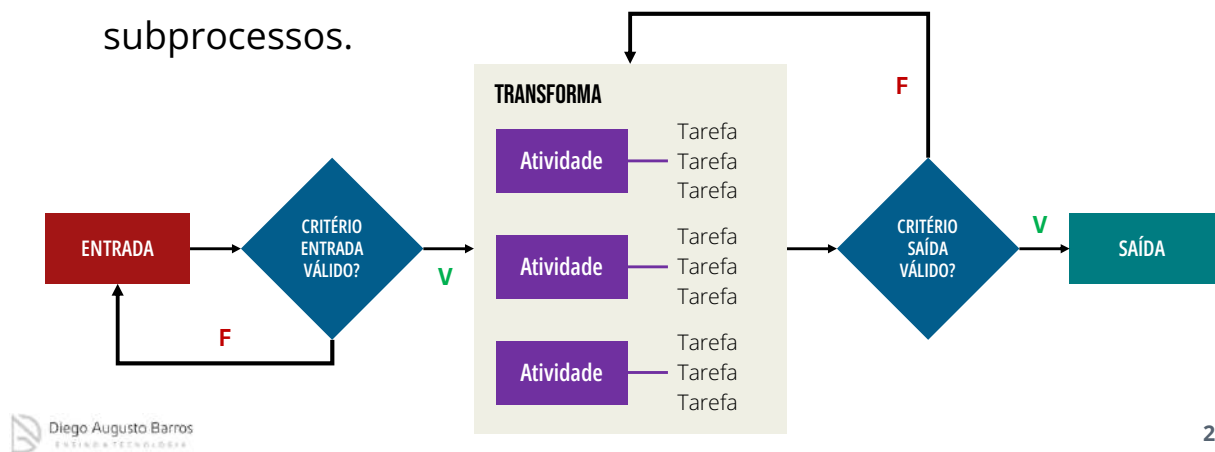
As atividades de trabalho podem ser subdivididas em tarefas, que são as unidades mais pequenas de trabalho sujeitas à responsabilidade de gestão.



23

ELEMENTOS DE UM PROCESSO DE SOFTWARE

Um processo de software pode incluir subprocessos.



24

EXEMPLO DE SUBPROCESSO – VALIDAÇÃO DE REQUISITOS

A **validação de requisitos** de software é um processo usado para determinar se os **requisitos** fornecerão uma base **adequada** para o desenvolvimento de software

VALIDAÇÃO DE
REQUISITOS

EXEMPLO DE SUBPROCESSO – VALIDAÇÃO DE REQUISITOS

A **validação de requisitos** de software é um é um **subprocesso** do processo de **requisitos** de software.

REQUISITOS DE
SOFTWARE

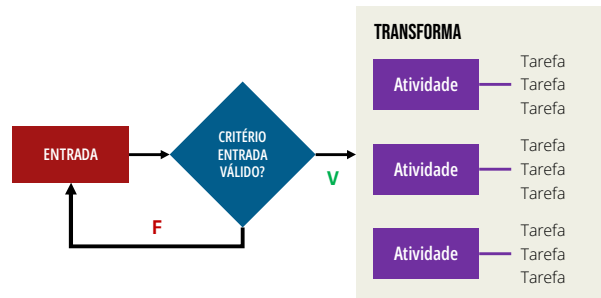


VALIDAÇÃO DE
REQUISITOS

EXEMPLO DE SUBPROCESSO – VALIDAÇÃO DE REQUISITOS

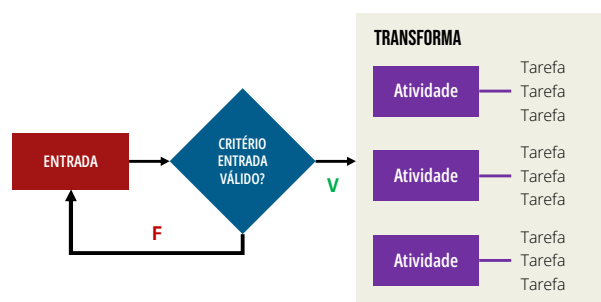
As entradas para a validação de requisitos geralmente são:

- Uma especificação de requisitos de software;
- Os recursos necessários para realizar a validação: pessoal, ferramentas de validação, tempo suficiente.



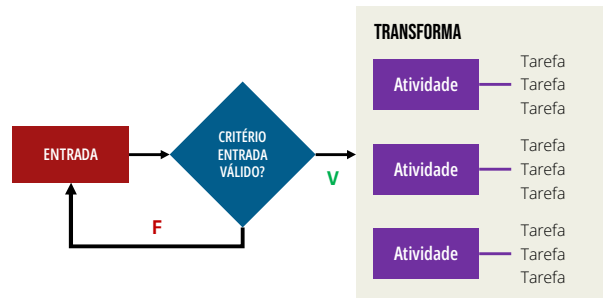
EXEMPLO DE SUBPROCESSO – VALIDAÇÃO DE REQUISITOS

- As tarefas da atividade de validação de requisitos podem incluir:
 - Revisões de requisitos;
 - Prototipagem;
 - Validação de modelos.
- Essas tarefas envolvem atribuições de trabalho para indivíduos e equipes.



EXEMPLO DE SUBPROCESSO – VALIDAÇÃO DE REQUISITOS

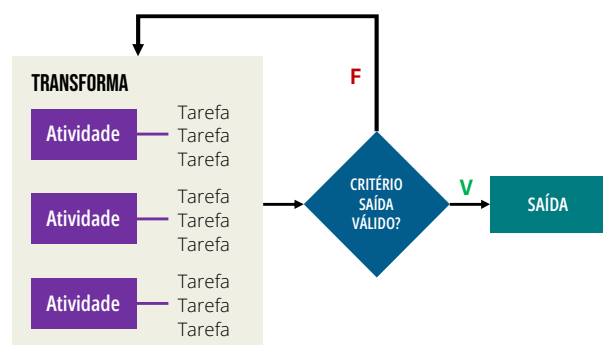
- As tarefas da atividade de validação de requisitos podem incluir:
 - Revisões de requisitos;
 - Prototipagem;
 - Validação de modelos.
- Essas tarefas envolvem atribuições de trabalho para indivíduos e equipes.



EXEMPLO DE SUBPROCESSO – VALIDAÇÃO DE REQUISITOS

A saída da validação de requisitos é tipicamente uma:

- Especificação de **Requisitos** de Software **Validada**: que fornece entradas para os processos de design e teste de software.



SUBPROCESSOS

- A validação de requisitos e outros **subprocessos** do processo de requisitos de software **são** frequentemente **intercalados** e **iterados** de várias maneiras.
- O processo de requisitos de software e seus **subprocessos podem ser iniciados e encerrados várias vezes** durante o desenvolvimento ou modificação de software.



PROCESSO DE SOFTWARE

A definição completa de um processo de software também pode incluir

1. Os papéis e competências;
2. Suporte de TI;
3. Técnicas e Ferramentas de Engenharia de Software;
4. Ambiente de Trabalho necessários para realizar o processo;
5. Abordagens e medidas (Indicadores-Chave de Desempenho) usados para determinar a eficiência e eficácia da execução do processo.

IMPORTANTE

Não existe um melhor processo
de software ou conjunto de
processos de software.

IMPORTANTE

Os processos de software devem ser
selecionados, adaptados e aplicados
conforme apropriado para cada projeto
e contexto organizacional.

IMPORTANTE

Não existe um processo ou conjunto de processos ideais.

ATIVIDADES FUNDAMENTAIS DO PROCESSO DE SOFTWARE



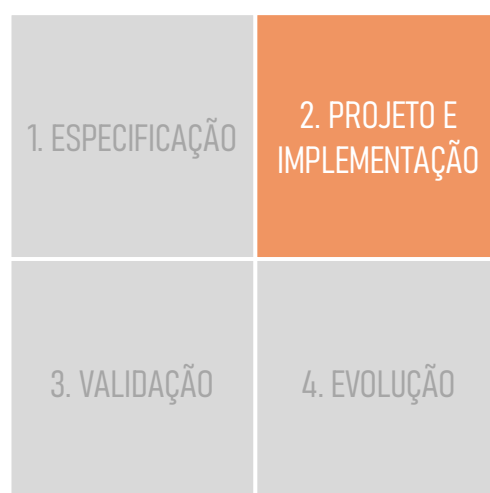
ATIVIDADES FUNDAMENTAIS DO PROCESSO DE SOFTWARE

A funcionalidade do software e as restrições a seu funcionamento devem ser definidas.



ATIVIDADES FUNDAMENTAIS DO PROCESSO DE SOFTWARE

O software deve ser produzido para atender às especificações.



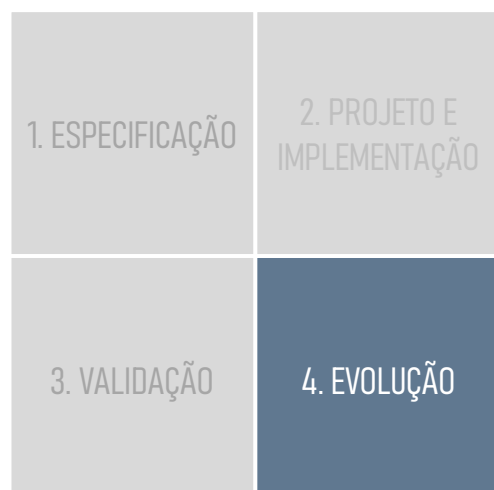
ATIVIDADES FUNDAMENTAIS DO PROCESSO DE SOFTWARE

O software deve ser validado para garantir que atenda às demandas do cliente.



ATIVIDADES FUNDAMENTAIS DO PROCESSO DE SOFTWARE

O software deve evoluir para atender às necessidades de mudança dos clientes



ATIVIDADES FUNDAMENTAIS DO PROCESSO DE SOFTWARE

De alguma forma, essas atividades fazem parte de todos os processos de software.



ATIVIDADES FUNDAMENTAIS DO PROCESSO DE SOFTWARE

Na prática, são atividades complexas em si mesmas, que incluem subatividades como:

- Validação de requisitos
- Projeto de Arquitetura
- Testes Unitários etc.



ATIVIDADES FUNDAMENTAIS DO PROCESSO DE SOFTWARE

Existem também as **atividades** que dão **apoio ao processo**, com:

- Documentação
- Gerenciamento de configuração de software.



DESCRIÇÕES DO PROCESSO DE SOFTWARE

As descrições do processo também podem incluir:

1. Produtos
2. Papéis
3. Pré e pós-condições

1. PRODUTOS

Produtos, são os **resultados** de uma das **atividades** do processo.



O **resultado** da atividade de **projeto de arquitetura** pode ser um **modelo da arquitetura** de software.

2. PAPÉIS

Os Papéis refletem as **responsabilidades das pessoas** envolvidas no processo, por exemplo:

- Gerente de Projeto,
- Gerente de Configuração,
- Programador etc.



3. PRÉ E PÓS-CONDIÇÕES

Pré e pós-condições, são **declarações verdadeiras** antes e depois de uma atividade do processo ou da produção de um produto.

Exemplo: antes do **projeto de arquitetura** ser iniciado, pode haver uma:

- **Pré-condição:** todos os requisitos tenham sido aprovados pelo cliente;
- **Pós-condição:** os modelos UML que descrevem a arquitetura tenham sido revisados.

2

FRAMEWORK DE PROCESSO

O FRAMEWORK DE PROCESSO

- Arcabouço (*Framework*) genérico que estabelece a **base** para o **processo** completo de engenharia de software.
- Identifica um conjunto **limitado** de **atividades** estruturais aplicáveis a **todos** os projetos de software.
- Independentemente do tamanho ou complexidade.

1. COMUNICAÇÃO

2. PLANEJAMENTO

3. MODELAGEM

4. CONSTRUÇÃO

5. IMPLANTAÇÃO

O FRAMEWORK DE PROCESSO

- Aborda aspectos comuns a diferentes projetos
- Engloba cinco atividades principais na engenharia de software.

1. COMUNICAÇÃO

2. PLANEJAMENTO

3. MODELAGEM

4. CONSTRUÇÃO

5. IMPLANTAÇÃO

O FRAMEWORK DE PROCESSO

Antes do início do trabalho técnico, é essencial comunicar e colaborar com o cliente e outros interessados.

1. COMUNICAÇÃO

2. PLANEJAMENTO

3. MODELAGEM

4. CONSTRUÇÃO

5. IMPLANTAÇÃO

O FRAMEWORK DE PROCESSO

O objetivo é compreender os objetivos do projeto e reunir requisitos para definir as características e funções do software.

1. COMUNICAÇÃO

2. PLANEJAMENTO

3. MODELAGEM

4. CONSTRUÇÃO

5. IMPLANTAÇÃO

O FRAMEWORK DE PROCESSO

A atividade de planejamento cria um "mapa" para orientar a equipe durante a jornada do projeto de software.

1. COMUNICAÇÃO

2. PLANEJAMENTO

3. MODELAGEM

4. CONSTRUÇÃO

5. IMPLANTAÇÃO

O FRAMEWORK DE PROCESSO

O plano do projeto de software define:

- Tarefas Técnicas;
- Riscos;
- Recursos Necessários;
- Produtos de Trabalho;
- Cronograma.

1. COMUNICAÇÃO

2. PLANEJAMENTO

3. MODELAGEM

4. CONSTRUÇÃO

5. IMPLANTAÇÃO

O FRAMEWORK DE PROCESSO

Assim como em outras áreas profissionais, os engenheiros de software utilizam modelos para compreender os requisitos e o design do software.

1. COMUNICAÇÃO

2. PLANEJAMENTO

3. MODELAGEM

4. CONSTRUÇÃO

5. IMPLANTAÇÃO

O FRAMEWORK DE PROCESSO

Modelos, semelhantes a esboços, ajudam a entender arquitetura, partes constituintes e detalhes do problema.

1. COMUNICAÇÃO

2. PLANEJAMENTO

3. MODELAGEM

4. CONSTRUÇÃO

5. IMPLANTAÇÃO

O FRAMEWORK DE PROCESSO

A atividade de construção envolve a geração de código (manual ou automatizada) e os testes necessários para identificar erros no código.

1. COMUNICAÇÃO

2. PLANEJAMENTO

3. MODELAGEM

4. CONSTRUÇÃO

5. IMPLANTAÇÃO

O FRAMEWORK DE PROCESSO

É a etapa em que o que foi projetado é efetivamente construído, dando vida ao software.

1. COMUNICAÇÃO

2. PLANEJAMENTO

3. MODELAGEM

4. CONSTRUÇÃO

5. IMPLANTAÇÃO

O FRAMEWORK DE PROCESSO

A entrega do software, seja como uma entidade completa ou como um incremento parcial, é realizada ao cliente.

1. COMUNICAÇÃO

2. PLANEJAMENTO

3. MODELAGEM

4. CONSTRUÇÃO

5. IMPLANTAÇÃO

O FRAMEWORK DE PROCESSO

O cliente avalia o produto entregue e fornece feedback com base na avaliação.

1. COMUNICAÇÃO

2. PLANEJAMENTO

3. MODELAGEM

4. CONSTRUÇÃO

5. IMPLANTAÇÃO

ATIVIDADES DO FRAMEWORK DE PROCESSO

Em muitos projetos de software, atividades do framework são aplicadas iterativamente durante o progresso do projeto.

1. COMUNICAÇÃO

2. PLANEJAMENTO

3. MODELAGEM

4. CONSTRUÇÃO

5. IMPLANTAÇÃO

ATIVIDADES DO FRAMEWORK DE PROCESSO

As atividades são aplicadas repetidamente ao longo de várias iterações do projeto.

1. COMUNICAÇÃO

2. PLANEJAMENTO

3. MODELAGEM

4. CONSTRUÇÃO

5. IMPLANTAÇÃO

ATIVIDADES DO FRAMEWORK DE PROCESSO

Cada iteração gera um incremento de software, oferecendo aos interessados um subconjunto das características e funcionalidades globais do software.

1. COMUNICAÇÃO

2. PLANEJAMENTO

3. MODELAGEM

4. CONSTRUÇÃO

5. IMPLANTAÇÃO

ATIVIDADES DO FRAMEWORK DE PROCESSO

À medida que cada incremento é produzido, o software se torna mais completo

1. COMUNICAÇÃO

2. PLANEJAMENTO

3. MODELAGEM

4. CONSTRUÇÃO

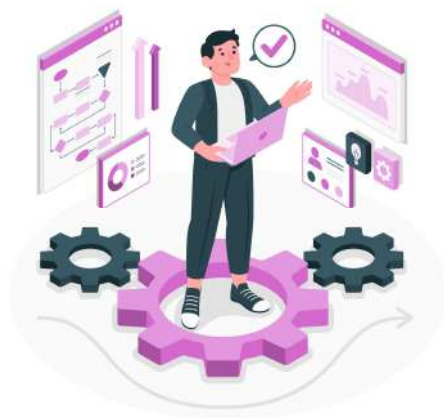
5. IMPLANTAÇÃO

ATIVIDADES DE APOIO

1. Controle e Acompanhamento do Projeto de Software
2. Gerência de Riscos
3. Garantia de Qualidade de Software
4. Revisões Técnicas
5. Medição
6. Gerência de Configuração de Software
7. Gestão de Reutilização
8. Preparação e Produção de Artefatos de Software

CONTROLE E ACOMPANHAMENTO DO PROJETO DE SOFTWARE

Permite à equipe de software **avaliar o progresso** em relação ao plano do projeto e tomar as medidas necessárias para manter o cronograma.



GERÊNCIA DE RISCOS

Avalia os **riscos** que podem **afetar** o **resultado** do projeto ou a qualidade do produto.



GARANTIA DE QUALIDADE DE SOFTWARE

Define e realiza atividades necessárias para garantir a **qualidade** do software.



REVISÕES TÉCNICAS

Avaliam os produtos de trabalho de engenharia de software para **descobrir e corrigir erros** antes que se propaguem para a próxima atividade.



MEDIÇÃO

Define e coleta **medidas** de **processo**, **projeto** e **produto** que ajudam a equipe a fornecer software que atenda às necessidades dos interessados;

Pode ser usada em conjunto com todas as outras atividades do framework e abrangentes.



GERÊNCIA DE CONFIGURAÇÃO DE SOFTWARE

Gerencia os efeitos da **mudança** ao longo do processo de software.



GESTÃO DE REUTILIZAÇÃO

Define critérios para a reutilização de produtos de trabalho (incluindo componentes de software)

Estabelece mecanismos para alcançar **componentes reutilizáveis**.



PREPARAÇÃO E PRODUÇÃO DE ARTEFATOS DE SOFTWARE

Engloba as atividades necessárias para criar produtos de trabalho, como:

- Modelos;
- Documentos;
- Registros;
- Formulários e Listas.



3

MODELOS DE PROCESSO DE SOFTWARE

MODELO DE PROCESSO DE SOFTWARE

Um modelo de processo de software é uma **representação** simplificada de um **processo** de software.



MODELO DE PROCESSO DE SOFTWARE

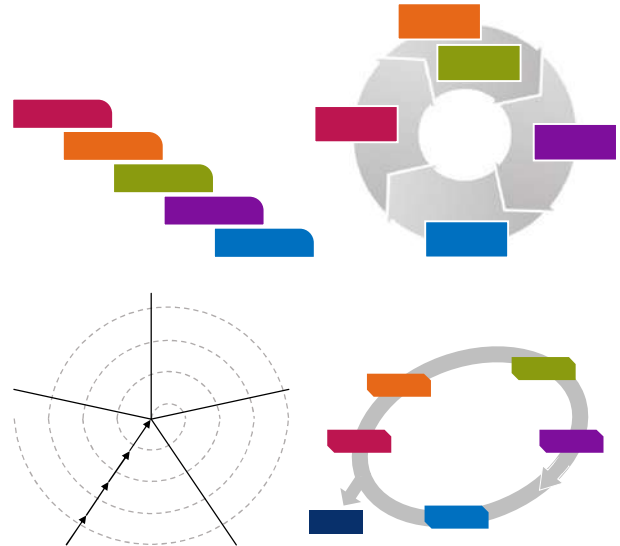
Cada **modelo** representa uma **perspectiva particular** de um processo e, portanto, fornece **informações parciais** sobre ele.

- Exemplo: um **modelo de atividade** do processo pode mostrar as **atividades e sua sequência**, mas não mostrar os papéis das pessoas envolvidas.



MODELOS DE SOFTWARE PRESCRITIVOS (TRADICIONAIS)

1. O Modelo Cascata
2. Modelo de Processo de Prototipação
3. Modelo de Processo Evolucionário
4. Modelo de Processo Unificado



O MODELO EM CASCATA

1. COMUNICAÇÃO

Início do Projeto
Levantamento de Requisitos

2. PLANEJAMENTO

Estimativas
Cronogramas
Acompanhamento

3. MODELAGEM

Análise
Projeto

4. CONSTRUÇÃO

Código
Testes

5. IMPLANTAÇÃO

Entrega
Suporte
Feedback

MODELO EM CASCATA

- Proposto por Winston Royce em 1970;
- Foi o primeiro modelo do processo de desenvolvimento de software a ser publicado;
- Derivado de processos mais gerais da engenharia de sistemas



Winston W. Royce (1929 – 1995)
Cientista da Computação

MODELO EM CASCATA

1. COMUNICAÇÃO

Início do Projeto
Levantamento de Requisitos

2. PLANEJAMENTO

Estimativas
Cronogramas
Acompanhamento

3. MODELAGEM

Análise
Projeto

4. CONSTRUÇÃO

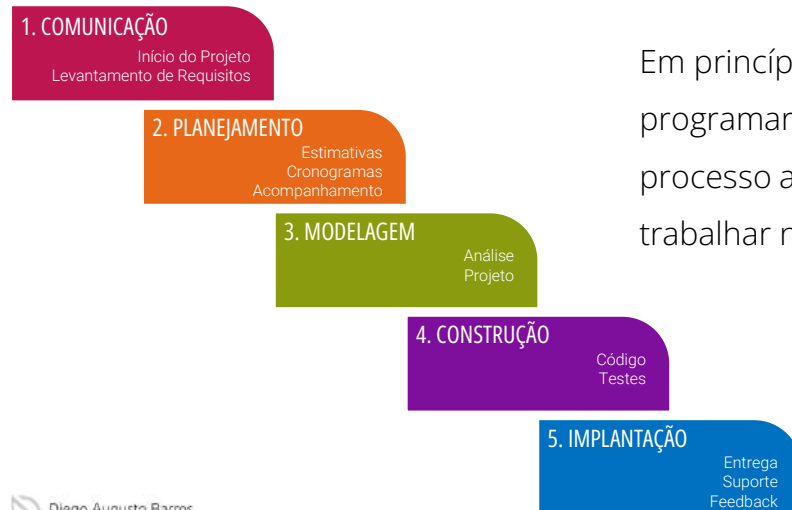
Código
Testes

5. IMPLANTAÇÃO

Entrega
Suporte
Feedback

Por causa do **encadeamento entre uma fase e outra**, esse modelo é conhecido como Modelo em Cascata, ou ciclo de vida de software.

MODELO EM CASCATA

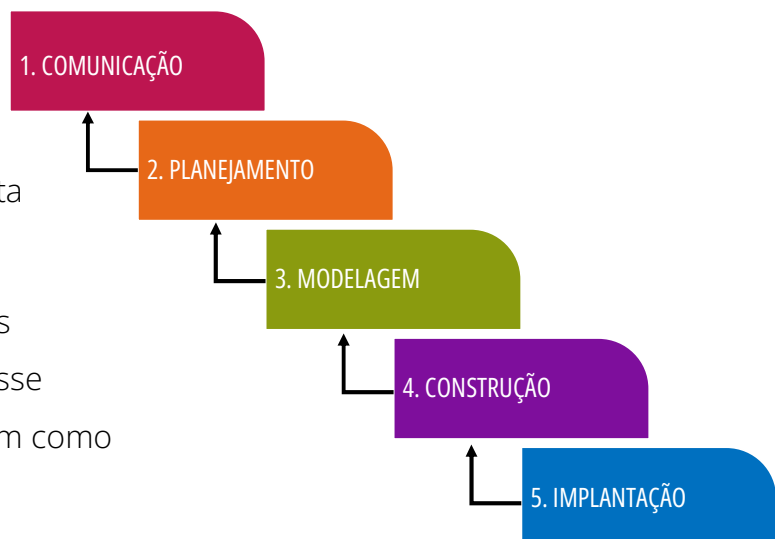


O modelo em cascata é um exemplo de um **processo dirigido a planos**

Em princípio, você deve planejar e programar todas as atividades do processo antes de começar a trabalhar nelas.

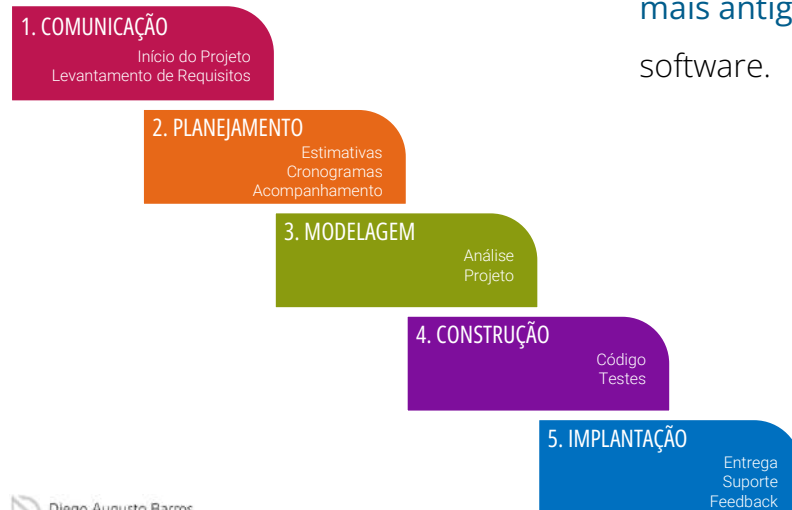
MODELO EM CASCATA

- O modelo original em cascata previa "ciclos de feedback"
- Porém, a grande maioria das organizações que aplicam esse modelo de processo o tratam como se fosse estritamente linear.



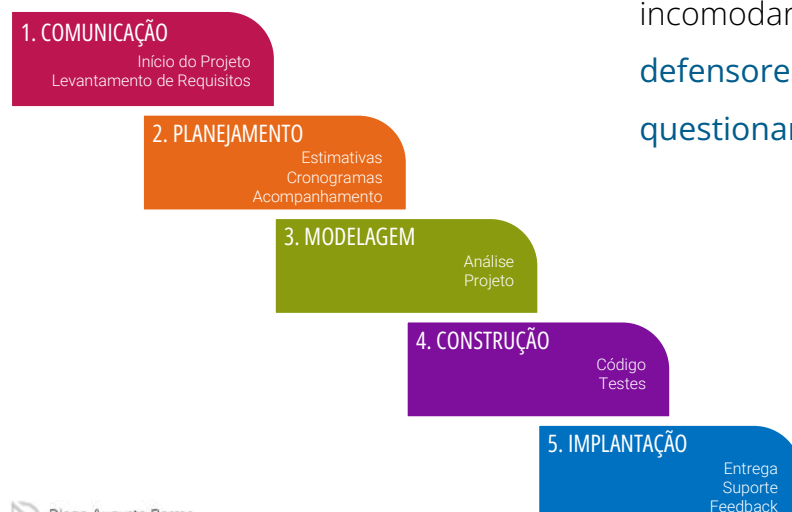
MODELO EM CASCATA

O modelo em cascata é o **paradigma mais antigo** para engenharia de software.



MODELO EM CASCATA

As críticas a este modelo de processo incomodaram **até mesmo aos defensores** mais fervorosos **questionamentos** sobre sua **eficácia**.



PROBLEMAS DO MODELO EM CASCATA

1. Projetos reais raramente seguem o fluxo de **trabalho sequencial** proposto pelo modelo.
2. Muitas vezes, é **difícil** para o cliente declarar todas as exigências explicitamente no início da maioria dos projetos.



PROBLEMAS DO MODELO EM CASCATA

3. O cliente deve ter paciência, pois uma versão funcional do(s) programa(s) **não estará disponível** até mais tarde no período do projeto.
4. Grandes **equivocos** podem não ser detectados até que o programa funcional seja revisado.



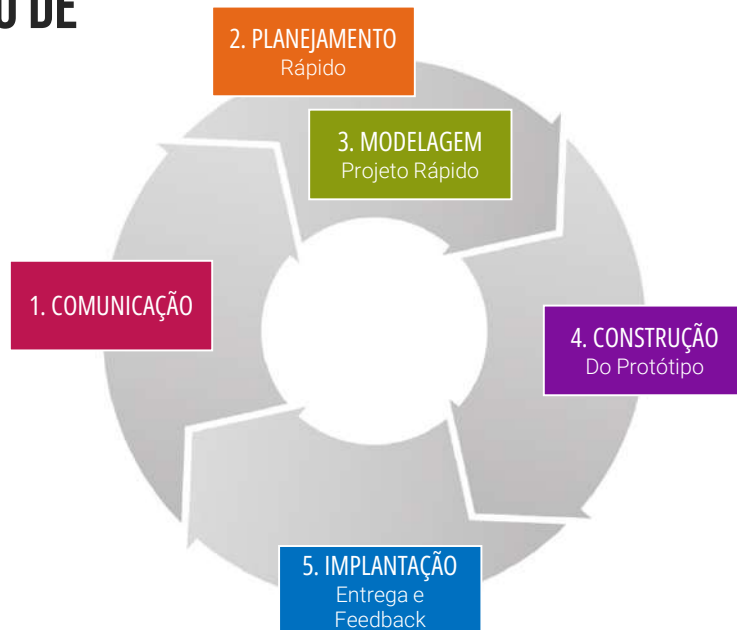
IMPORTANTE

Hoje, o trabalho com software tem um **ritmo acelerado** e está sujeito a um fluxo contínuo de **mudanças** em termos de características, funções e conteúdo de informações.

IMPORTANTE

O Modelo em Cascata
frequentemente é **inadequado**
para esse tipo de trabalho.

MODELO DE PROCESSO DE PROTOTIPAÇÃO



PROTOTIPAGEM

A prototipagem fornece uma **manifestação concreta de uma ideia**, seja um produto novo ou uma modificação de um já existente.



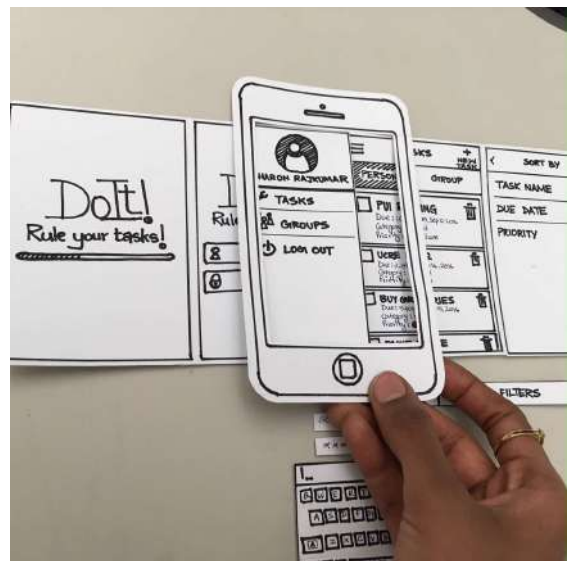
PROTOTIPAGEM

Ela permite que os **designers** **comuniquem** suas ideias e que os **usuários** as **experimentem**.



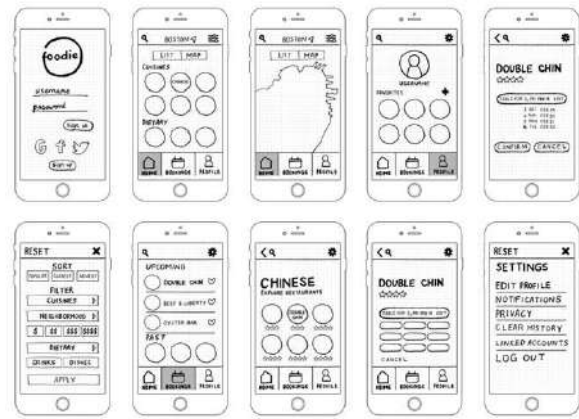
O QUE É UM PROTÓTIPO?

Um protótipo é uma **manifestação de um design** que permite às partes interessadas (*Stakeholders*) interagir com ele e explorar sua adequação.



O QUE É UM PROTÓTIPO?

Nos **estágios iniciais** do desenvolvimento, esses protótipos podem ser feitos de **papel**, **papelão** ou **componentes prontos** reunidos para permitir a avaliação.



O QUE É UM PROTÓTIPO?

À medida que o **projeto avança**, eles se tornam mais polidos, adaptados e robustos para **se parecerem com o produto final**.



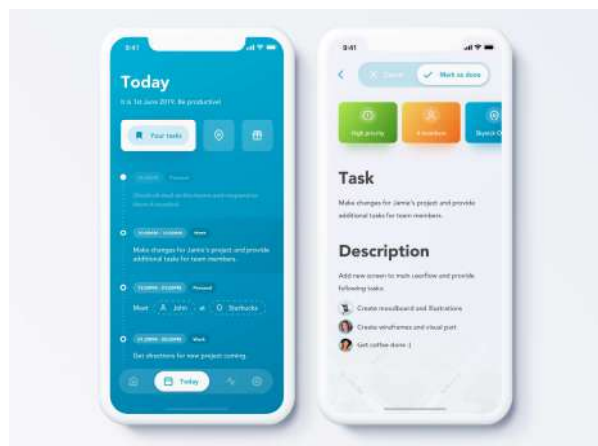
PROTÓTIPOS NO PROCESSO DE SOFTWARE

Um protótipo de software pode ser usado em um processo de desenvolvimento de software para ajudar a **antecipar as mudanças que podem ser requisitadas**

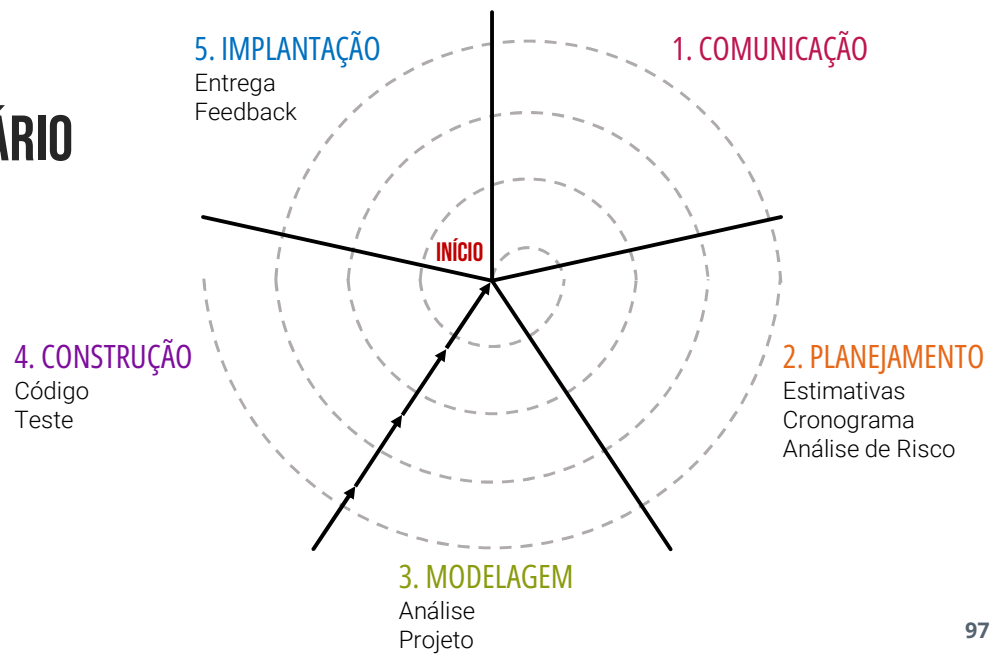


PROTÓTIPOS NO PROCESSO DE SOFTWARE

1. **Engenharia de Requisitos:** ajudar na elicitação e validação de requisitos de sistema.
2. **Projeto de Sistema:** estudar soluções específicas do software e para apoiar o projeto de interface de usuário.



MODELO DE PROCESSO EVOLUCIONÁRIO



MODELO ESPIRAL

Originalmente proposto
por Barry Boehm em 1988

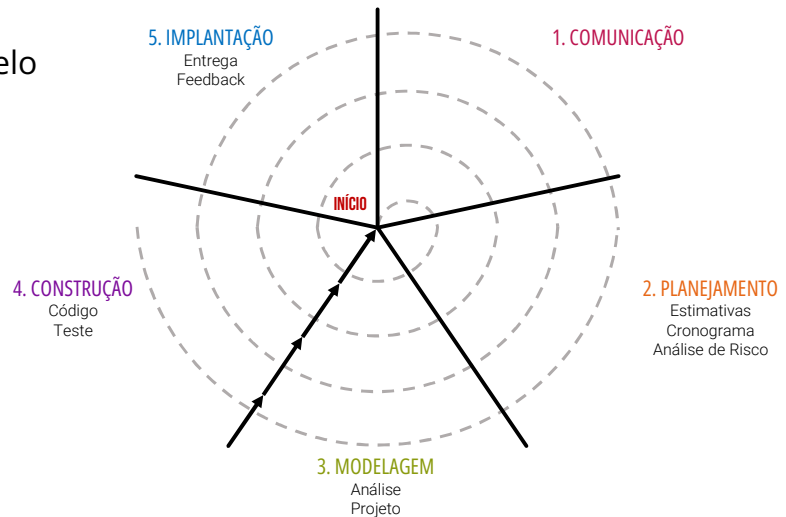


Barry Boehm (1935 – 2022)
Engenheiro de Software

MODELO ESPIRAL

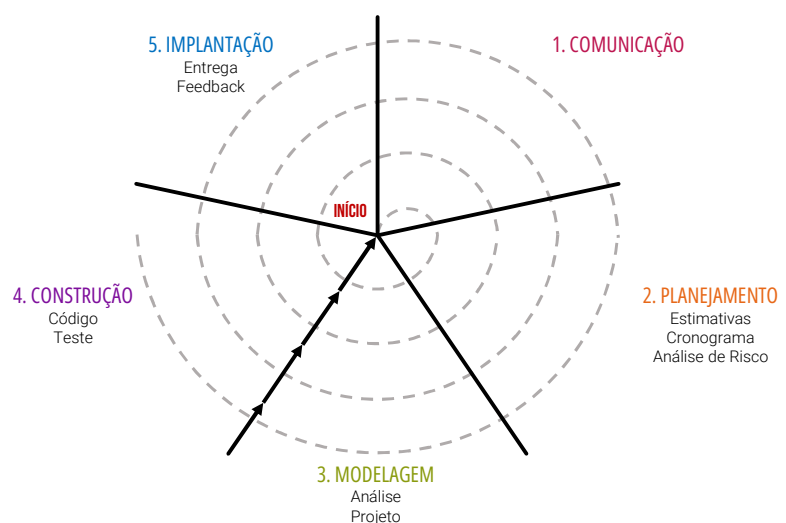
O modelo espiral é um modelo evolutivo de processo de software que **combina**:

1. A natureza iterativa de **prototipagem**;
2. Com os aspectos controlados e sistemáticos do modelo em **cascata**.



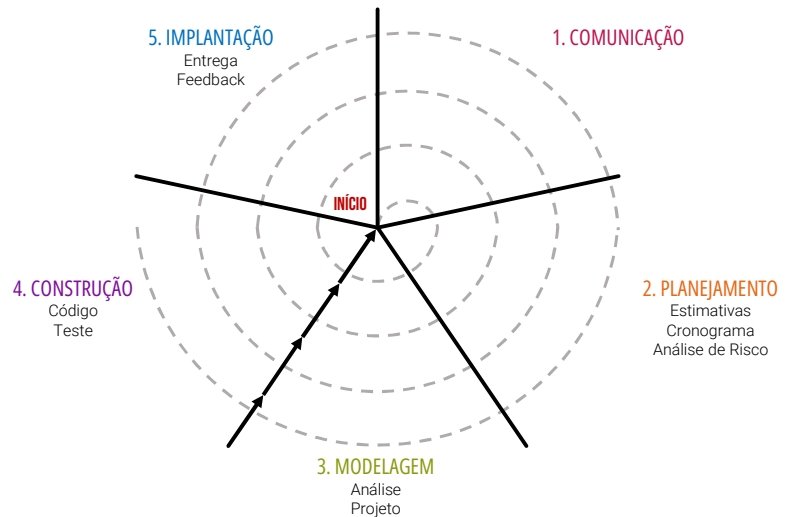
MODELO ESPIRAL

Ele oferece o **potencial** para o **desenvolvimento rápido** de versões cada vez mais completas do software.



MODELO ESPIRAL

Usando o modelo espiral, o software é desenvolvido em uma série de lançamentos evolutivos.



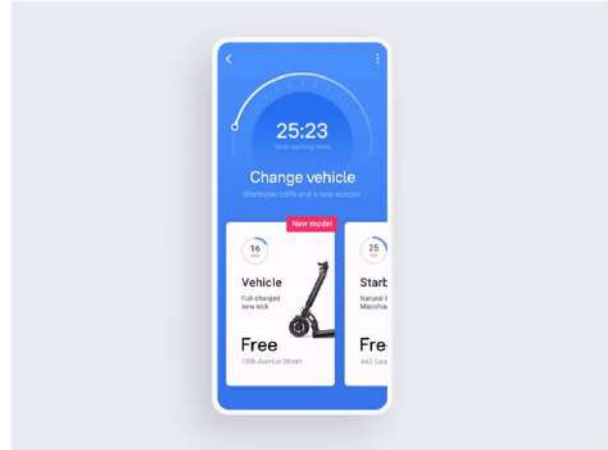
MODELO ESPIRAL

Durante as iterações iniciais, o lançamento pode ser um modelo ou protótipo.



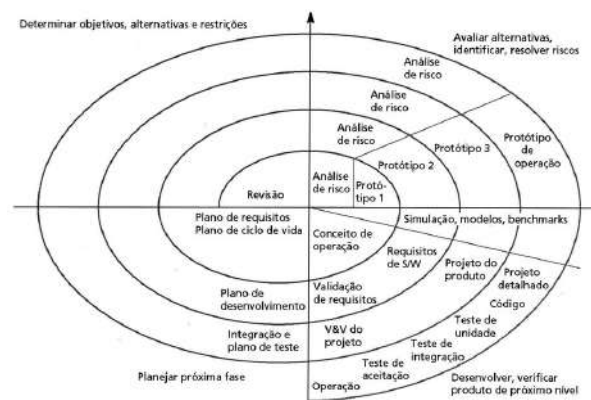
MODELO ESPIRAL

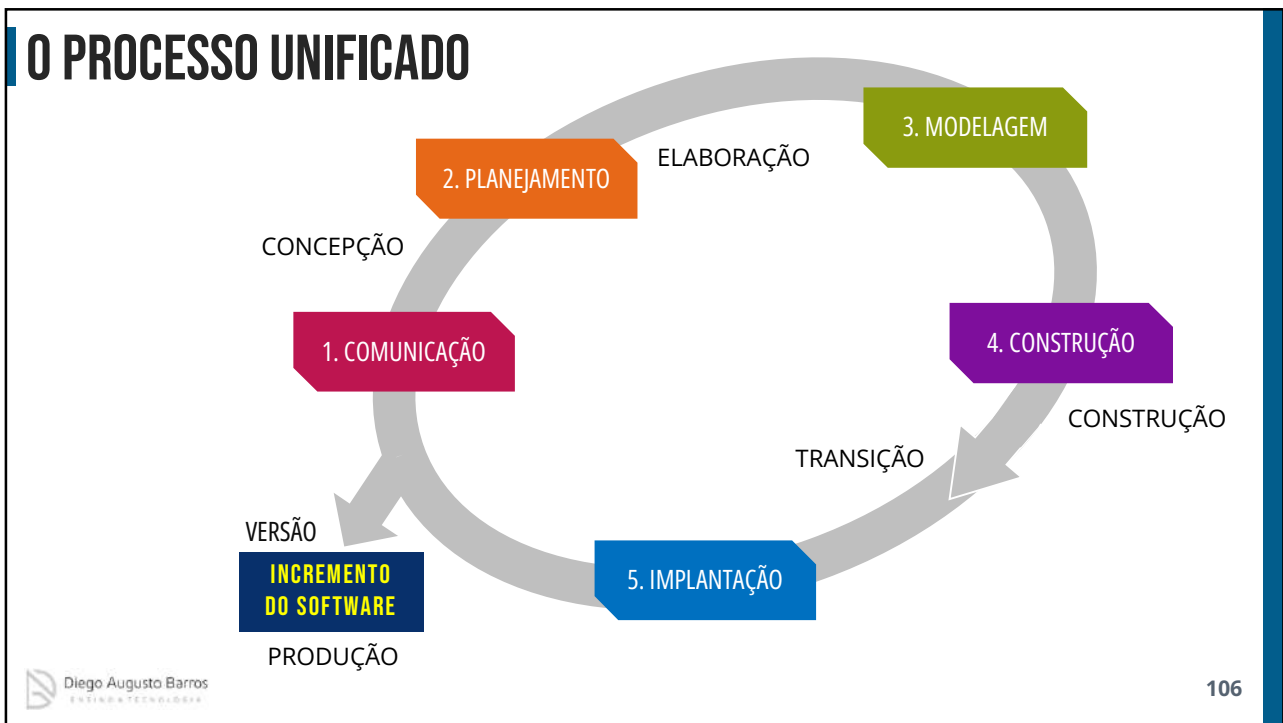
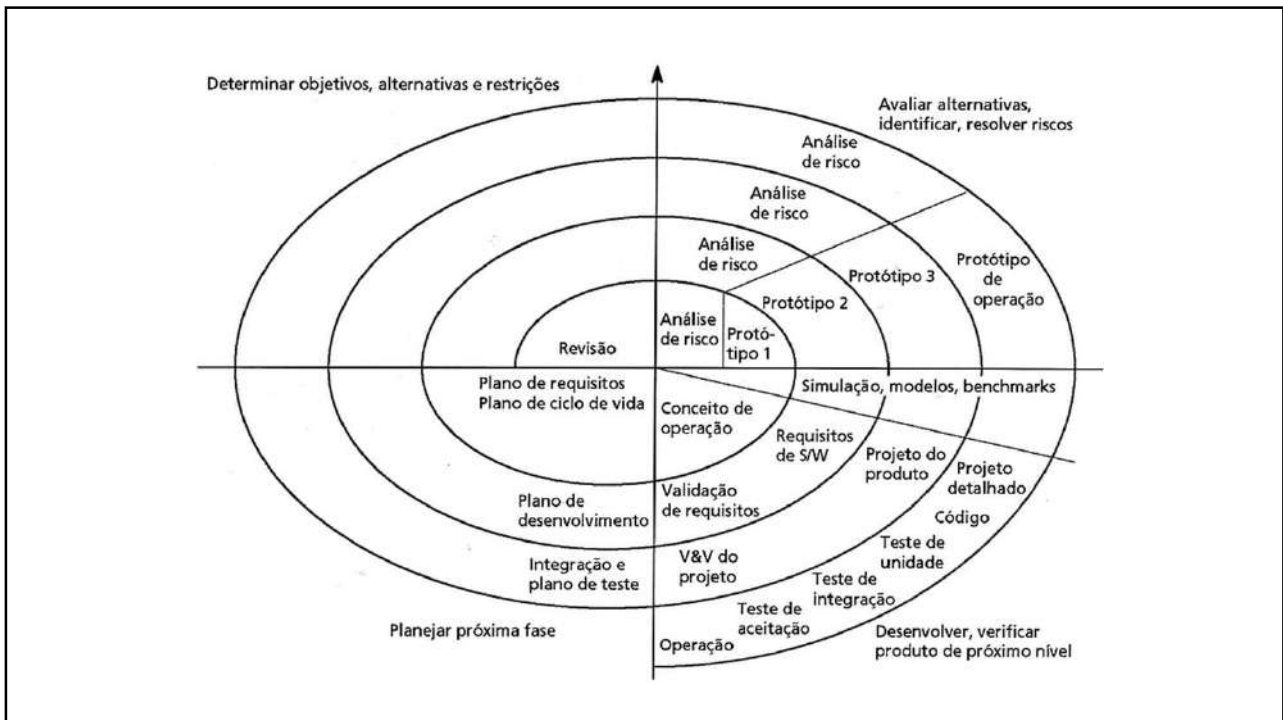
Durante **iterações** posteriores,
versões cada vez mais **completas**
do sistema desenvolvido são
produzidas.



MODELO ESPIRAL

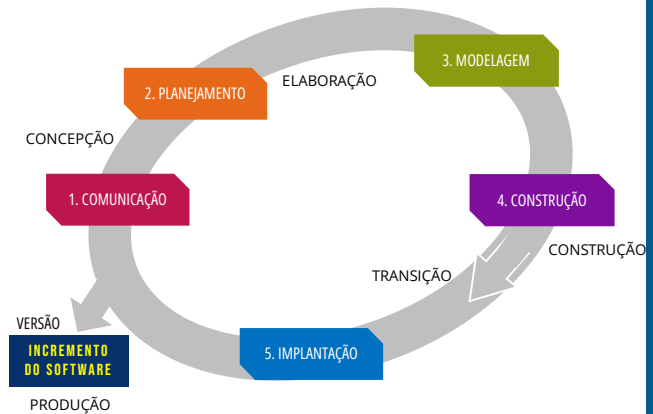
Um modelo espiral é **dividido** em um conjunto de **atividades** de estrutura **definidas** pela **equipe** de engenharia de software.





O PROCESSO UNIFICADO

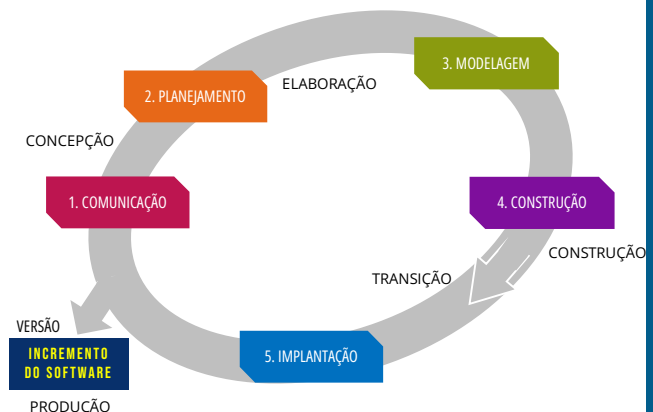
- Aproveitar as melhores **características** e **elementos** dos **modelos tradicionais** de processo de software,
- Mas caracterizá-los de uma maneira que **implementa** muitos dos melhores **princípios** do desenvolvimento **ágil** de software.



O PROCESSO UNIFICADO

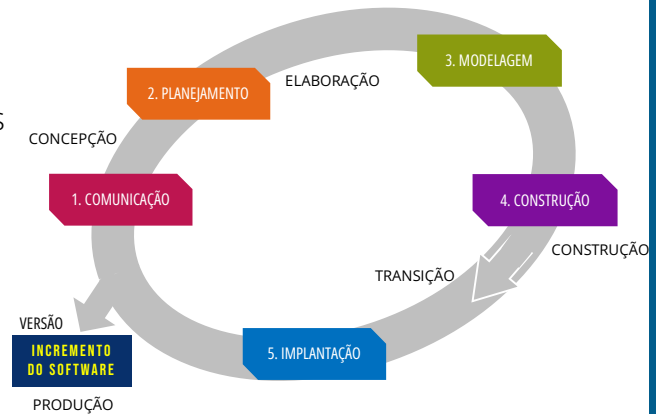
O Processo Unificado reconhece:

- A importância da **comunicação** com o **cliente**;
- Métodos simplificados para descrever a **visão do cliente** sobre um **sistema** (p.ex.: Casos de Uso).



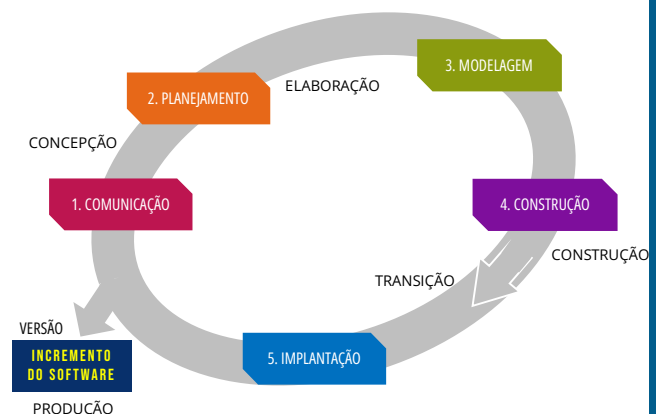
O PROCESSO UNIFICADO

- Enfatiza o papel crucial da arquitetura de software
- Ajuda o arquiteto a focar nos objetivos certos, como:
 - Compreensibilidade;
 - Confiabilidade, para futuras alterações e reutilização.



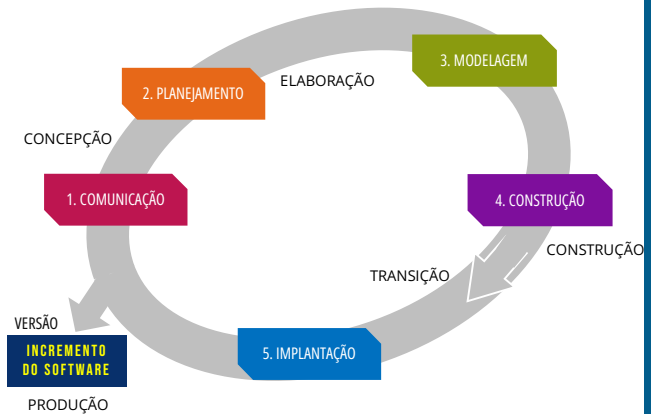
O PROCESSO UNIFICADO

- Ele sugere um fluxo de processo que é **iterativo** e **incremental**;
- Proporcionando a **sensação evolutiva**;
- Essencial no desenvolvimento moderno de software.



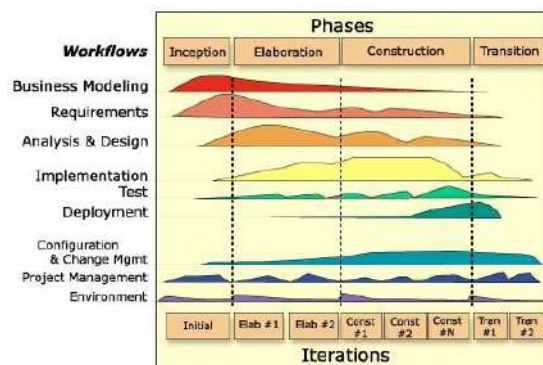
O PROCESSO UNIFICADO

A Linguagem de Modelagem Unificada (**UML**), foi desenvolvida para apoiar seu trabalho.



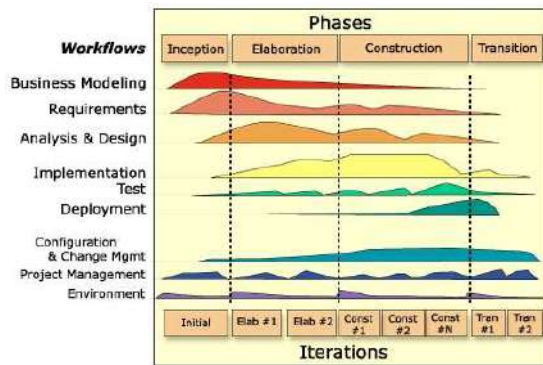
FASES DO PROCESSO UNIFICADO

1. **Concepção:** atingir o consenso entre todos os investidores sobre os objetivos do ciclo de vida do projeto;
2. **Elaboração:** Estabelecer a estrutura fundamental da arquitetura do sistema, visando proporcionar uma base sólida para a execução da fase de construção;



FASES DO PROCESSO UNIFICADO

3. **Construção:** esclarecer os requisitos restantes e concluir o desenvolvimento do sistema com base na arquitetura definida;
4. **Transição:** assegurar que o software esteja disponível a seus usuários



COMPARAÇÃO ENTRE OS MODELOS

MODELO	PRÓS	CONTRAS
Cascata	Fácil de entender e planejar. Funciona para projetos pequenos e bem compreendidos. A análise e o teste são simples e diretos.	Não se adapta a mudanças. O teste ocorre nas fases finais do processo. A aprovação do cliente vem no final.
Prototipação	O impacto das alterações aos requisitos é reduzido. O cliente se envolve bastante e desde o início. Funciona bem para projetos pequenos. A probabilidade de rejeição do produto é reduzida.	O envolvimento do cliente pode causar atrasos. Pode haver a tentação de "embalar" o protótipo. Desperdiça-se trabalho em um protótipo descartável. É difícil de planejar e gerenciar.

COMPARAÇÃO ENTRE OS MODELOS

MODELO	PRÓS	CONTRAS
Espiral	Há envolvimento contínuo dos clientes. Os riscos de desenvolvimento são gerenciados. É apropriado para modelos grandes e complexos. Funciona bem para artefatos extensíveis.	Falhas de análise de risco podem fadar o projeto ao fracasso. O projeto pode ser difícil de gerenciar. Exige uma equipe de desenvolvimento especializada.
Unificado	A documentação de alta qualidade é enfatizada. Há envolvimento contínuo dos clientes. Adapta-se a alterações aos requisitos. Funciona bem para projetos de manutenção.	Os casos de uso nem sempre são precisos. A integração de incrementos de software é complicada. A sobreposição das fases pode causar problemas. Exige uma equipe de desenvolvimento especializada.

MODELO DE PROCESSO DE SOFTWARE

Os modelos de processo foram originalmente propostos para **trazer ordem ao caos** do desenvolvimento de software.



MODELO DE PROCESSO DE SOFTWARE

A história indicou que esses modelos

- Trouxeram uma quantidade útil de **estrutura** para o trabalho de engenharia de software;
- Forneceram um **mapa** eficaz para equipes de software.



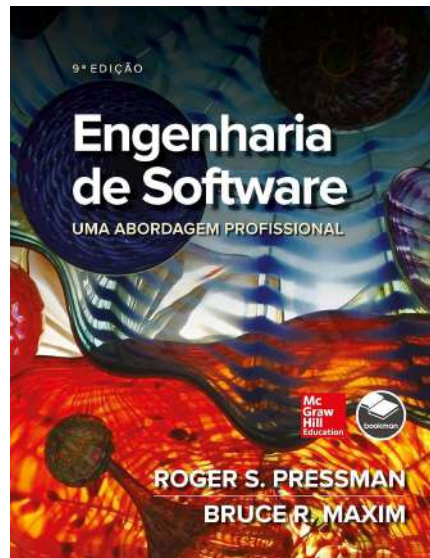
BIBLIOGRAFIA

SOMMERVILLE, Ian. **Engenharia de Software**. Cap. 2 – Processo de Software. pág. 18. Cap. 3 – Desenvolvimento Ágil de Software. pág. 38. Pearson, 2019.



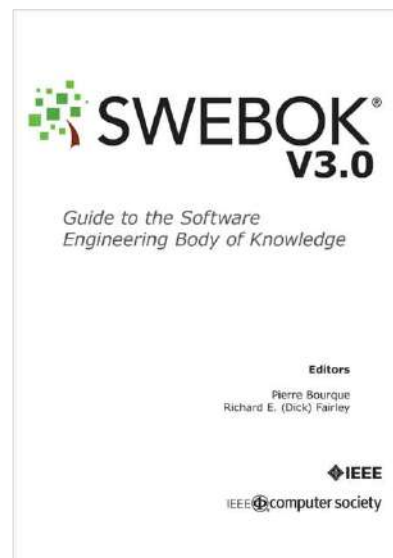
BIBLIOGRAFIA

PRESSMAN, Roger S.; MAXIM, Bruce R. **Engenharia de Software: Uma Abordagem Profissional**. Cap. 4.1 Modelos de Processos Prescritivos, pág. 41. AMGH, 2021.



BIBLIOGRAFIA

BOURQUE, Pierre.; FAIRLEY, Richard E. (Dick). **SWEBOK V3.0**. Chapter 8. Software Engineering Process, pág. 81. AMGH, 2021.





PROF. DIEGO AUGUSTO BARROS



diegoaugustobarros.com



@diegoaugustobarros



@profdiegoaugusto

Diego Augusto Barros é bacharel em Sistemas de Informação pela Pontifícia Universidade Católica de Minas Gerais (2012) e mestre em Ciência da Computação pela Universidade Federal de Minas Gerais (2015).

Sua pesquisa concentra-se nas áreas de Visualização de Dados e Interação Humano-computador, e investiga fatores cognitivos e perceptivos envolvidos na análise de grandes conjuntos de dados, que resultam em novos sistemas interativos para comunicação e análise visual.

Seus principais interesses nas áreas são: visualização de informação, Visual Analytics, métodos de avaliação de interfaces, interação com sistemas, tecnologias web, sistemas de informação, engenharia de software e informática na educação.